

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN
LƯU TRỮ VÀ XỬ LÝ DỮ LIỆU LỚN

Đề tài: Hệ thống giám sát giao thông công cộng

Giảng viên hướng dẫn: TS. Trần Việt Trung

Mã lớp: 168482

Họ và Tên	MSSV
Chu Văn An	20234998
Nguyễn Trọng Dũng	20235054
Nguyễn Đình Duy	20235064
Bùi Công Hoan	20235083
Nguyễn Hoàng Vũ	20235251

HÀ NỘI, 5/2026

LỜI CẢM ƠN

Mặc dù khoảng thời gian được làm việc dưới sự hướng dẫn của TS. Trần Việt Trung trong bộ môn Lưu trữ và xử lý dữ liệu lớn là không dài, nhưng chúng em đã học được nhiều kiến thức quý báu, không chỉ gói gọn trong bộ môn Lưu trữ và xử lý dữ liệu lớn mà còn có thể áp dụng vào nhiều công việc khác nhau trong tương lai.

Để đạt được những kết quả như vậy, chúng em xin chân thành cảm ơn thầy đã tận tình chỉ bảo, hướng dẫn, và truyền đạt những kiến thức quý báu cho em trong thời gian qua. Với lòng biết ơn chân thành, chúng em xin gửi lời chúc sức khỏe đến các thầy cô trong khoa, trong nhà trường, và đặc biệt là thầy Trần Việt Trung.

Nhóm sinh viên thực hiện

TÓM TẮT NỘI DUNG PROJECT

Hệ thống giao thông công cộng Singapore phát sinh lượng dữ liệu khổng lồ theo thời gian thực, bao gồm thông tin về hàng nghìn tuyến xe buýt, các ga MRT, bãi đỗ xe, trạm sạc EV và hàng trăm taxi đang hoạt động. Việc thu thập, lưu trữ và phân tích nguồn dữ liệu này đặt ra nhiều thách thức đối với các hệ thống xử lý dữ liệu truyền thống. Xuất phát từ yêu cầu đó, dự án Hệ thống giám sát giao thông công cộng được xây dựng nhằm mô phỏng một hệ thống Big Data hoàn chỉnh, có khả năng xử lý đồng thời dữ liệu tĩnh, dữ liệu on-demand và dữ liệu streaming theo thời gian thực.

Dự án áp dụng kiến trúc Lambda để kết hợp ưu điểm của hai mô hình xử lý batch và streaming. Hệ thống được thiết kế với ba luồng dữ liệu tách biệt:

- Static: Dữ liệu 5.000 trạm bus (tọa độ, tên) được tải một lần vào MongoDB, Frontend tải về khi mở web để vẽ bản đồ.
- On-demand: Khi user click vào trạm, FastAPI gọi trực tiếp LTA BusArrival API, trả về ETA lập tức mà không đi qua Kafka hay Spark.
- Big Data Background: Ingestor poll 50 trạm đồng nhất (30 giây/lần) đẩy vào Kafka → Spark Streaming xử lý → MinIO (raw Parquet) → Spark Batch train MLlib.

Hạ tầng được triển khai trên Minikube với MinIO làm object storage. Dữ liệu thời gian thực được đẩy qua Apache Kafka (Strimzi KRaft), xử lý bằng Spark Structured Streaming và lưu vào MongoDB để FastAPI phục vụ web app 5 có tab: Bus, MRT, Carpark, EV Charging, Taxi.

MỤC LỤC

PHÂN CÔNG THÀNH VIÊN TRONG NHÓM	6
1 Giới thiệu chung	7
1.1 Bối cảnh dự án	7
1.2 Mục tiêu của dự án	7
1.3 Phạm vi và giới hạn của dự án	8
1.4 Phương pháp tiếp cận và công nghệ sử dụng	8
2 Định nghĩa bài toán	9
2.1 Mô tả bài toán thực tế được lựa chọn	9
2.2 Đặc điểm dữ liệu và nguồn dữ liệu	9
2.3 Phân tích phù hợp với Dữ liệu lớn	10
2.3.1 Volume	10
2.3.2 Velocity	10
2.3.3 Variety	10
2.3.4 Veracity	10
2.3.5 Value	10
2.4 Các yêu cầu nghiệp vụ	10
2.5 Các yêu cầu kỹ thuật	11
2.6 Các giả định và ràng buộc	11
3 Tổng quan kiến trúc hệ thống	12
3.1 Tổng quan kiến trúc Dữ liệu lớn	12
3.2 Lựa chọn mô hình kiến trúc	12
3.2.1 Lambda Architecture	12
3.2.2 Kappa Architecture	13
3.2.3 So sánh và lý do lựa chọn Lambda	13
3.3 Luồng dữ liệu end-to-end	14
4 Thiết kế chi tiết hệ thống	15
4.1 Thiết kế tầng Data Ingestion	15
4.1.1 Nguồn dữ liệu và chiến lược thu thập dữ liệu	15
4.1.2 Message Queue	15
4.1.3 Chiến lược xử lý dữ liệu đến trễ và trùng lặp	16
4.2 Thiết kế tầng Data Processing với Apache Spark	16
4.2.1 Batch Processing (Xử lý dữ liệu định kỳ)	17
4.2.2 Stream Processing (Xử lý dữ liệu luồng)	17
4.2.3 Windowing & Watermarking (Cửa sổ thời gian và Lọc trễ)	17

4.2.4	Quản lý trạng thái	18
4.2.5	Cơ chế Exactly-Once	18
4.3	Thiết kế tầng Data Storage	18
4.3.1	MinIO - Object Storage	18
4.3.2	NoSQL Database	19
4.3.3	Cache Layer	20
4.3.4	Chiến lược phân vùng và nén	20
4.4	Thiết kế tầng Analytics & Visualization	21
4.5	Thiết kế tầng Monitoring & Logging	21
5	Triển khai hệ thống	23
5.1	Kiến trúc môi trường và Phân bổ tài nguyên	23
5.2	Chiến lược và quy trình triển khai	24
5.3	Cấu hình và Bảo mật hệ thống	24
6	Xử lý dữ liệu với Apache Spark	25
6.1	Kiến trúc luồng xử lý dữ liệu	25
6.2	Kỹ thuật chuyển đổi dữ liệu	25
6.2.1	Chuỗi chuyển đổi đa giai đoạn	25
6.2.2	Tối ưu hóa chuỗi thực thi	26
6.2.3	Các hàm tự định nghĩa	26
6.3	Kỹ thuật tổng hợp dữ liệu	26
6.4	Tối ưu hóa các phép kết nối	27
6.4.1	Phép kết nối quảng bá	27
6.4.2	Phép kết nối cấu trúc đồ thị	27
6.5	Các kỹ thuật tối ưu hóa hiệu năng	27
7	Xử lý dữ liệu theo thời gian thực	29
7.1	Kiến trúc Structured Streaming	29
7.2	Chiến lược ghi dữ liệu	30
7.3	Kỹ thuật xử lý dữ liệu trễ	30
7.4	Quản trị trạng thái	31
7.5	Hiện thực hóa Khái niệm Exactly-once	31
8	Phân tích Cấu trúc Đồ thị Mạng lưới Giao thông với GraphFrames	32
8.1	Mục tiêu và vai trò của Phân tích Đồ thị trong hệ thống	32
8.2	Cấu trúc của dữ liệu đồ thị và kỹ thuật trích xuất	32
8.3	Thực thi thuật toán PageRank	33
8.4	Phân tích cấu trúc đồ thị với GraphFrames	33
9	Đánh giá hệ thống	34
9.1	Đánh giá hiệu năng hệ thống	34
9.2	Độ ổn định và khả năng mở rộng	35
10	Bài học kinh nghiệm	36
10.1	Tầm quan trọng của Quản trị tài nguyên	36
10.2	Sự đánh đổi trong thiết kế kiến trúc	36
10.3	Tính toàn vẹn trong dữ liệu thời gian thực	36

10.4 Tư duy xây dựng hệ thống thực tiễn	36
11 Tổng kết và định hướng phát triển	38
11.1 Tổng kết kết quả đã đạt được	38
11.2 Hạn chế của hệ thống	38
11.3 Hướng phát triển trong tương lai	39
KẾT LUẬN	40
TÀI LIỆU THAM KHẢO	41
PHỤ LỤC	43
A MÃ NGUỒN DỰ ÁN	43
B CÂY THƯ MỤC DỰ ÁN	43
C DANH SÁCH BUS_HOT_STOPS	44

PHÂN CÔNG THÀNH VIÊN TRONG NHÓM

Họ và tên	Tổng hợp công việc thực hiện	Đánh giá
Chu Văn An	Hoàn thiện Dashboard, báo cáo	Tốt
Nguyễn Trọng Dũng	Thiết kế Batch	Tốt
Nguyễn Đình Duy	Lấy dữ liệu, lập trình để tự động dữ liệu về Kafka	Tốt
Bùi Công Hoan	Streaming dữ liệu bằng Spark	Tốt
Nguyễn Hoàng Vũ	Thiết lập môi trường, api, xây dựng Minikube, MongoDB,...	Tốt

Chương 1

Giới thiệu chung

1.1 Bối cảnh dự án

Singapore là một trong những quốc gia có hệ thống giao thông công cộng phát triển và được số hóa hoàn toàn tại châu Á. Cơ quan Giao thông đường bộ Singapore (LTA) cung cấp nền tảng dữ liệu mở LTA DataMall với hơn 20 API endpoints cập nhật theo thời gian thực, bao gồm thông tin về hơn 5.000 trạm xe buýt, các tuyến MRT, bãi đỗ xe, trạm sạc EV và vị trí taxi đang rảnh. Đây là nguồn dữ liệu phong phú và hiếm có trong khu vực. Đặc biệt nguồn dữ liệu này hoàn toàn được miễn phí và công khai. Tạo điều kiện lý tưởng để xây dựng một hệ thống dữ liệu lớn có tính thực tiễn cao.

Xuất phát từ bối cảnh đó, dự án hướng đến việc xây dựng một ứng dụng giám sát giao thông đa phương tiện, cho phép người dùng tra cứu thông tin thời gian thực về xe buýt, MRT, bãi đỗ xe, trạm sạc EV và taxi trong một giao diện thống nhất, đồng thời vận hành một pipeline Big Data hoàn chỉnh theo kiến trúc Lambda.

1.2 Mục tiêu của dự án

Mục tiêu chính của dự án là xây dựng một pipeline Big Data end-to-end cho dữ liệu giao thông công cộng Singapore, đáp ứng các yêu cầu về thu thập, xử lý, lưu trữ và trực quan hóa dữ liệu. Cụ thể, dự án hướng tới các mục tiêu sau:

- Xây dựng hệ thống thu thập dữ liệu giao thông từ LTA DataMall API với độ tin cậy cao, áp dụng chiến lược On-Demand thay vì poll toàn bộ để tối ưu tài nguyên.
- Thiết kế kiến trúc Lambda với ba luồng dữ liệu tách biệt: static, on-demand và streaming background.
- Ứng dụng các công nghệ cốt lõi của hệ sinh thái Big Data: Apache Spark, Kafka (Strimzi), MongoDB, Redis, MinIO.
- Triển khai toàn bộ hạ tầng trên Minikube.
- Cung cấp web app có 5 tab với bản đồ tương tác (Leaflet.js) để theo dõi giao thông theo thời gian thực.

1.3 Phạm vi và giới hạn của dự án

Dự án của chúng em tập trung vào việc:

- Thu thập và xử lý dữ liệu từ 6 endpoints của api từ LTA: BusArrival, BusStops, MRT Crowd, CarParkAvailability, EVCBatch, Taxi-Availability.
- Xây dựng pipeline streaming xử lý 5 Kafka topics với window 5 phút.
- Serving layer với FastAPI và web app React và Leaflet.js.
- Triển khai hạ tầng của ứng dụng trên Minikube với MinIO.

1.4 Phương pháp tiếp cận và công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Hạ tầng	Minikube (Kubernetes Local)	Trục xương sống kết nối hệ thống
Object Storage	MinIO	Data Lake lưu trữ dữ liệu thô
Message Queue	Apache Kafka	Là vùng đệm dữ liệu, phân phối dữ liệu cho hệ thống
Xử lý Streaming	Spark Structured Streaming	Speed Layer
Xử lý Batch	PySpark, MLlib, GraphFrames	Batch Layer, xây dựng mô hình dự báo, tính toán PageRank
NoSQL	MongoDB	Lưu trữ dữ liệu tĩnh, speed/ batch views
Serving	FastAPI	Cầu nối giữa dữ liệu và giao diện
Frontend	HTML và Leaflet.js	Tạo giao diện với bản đồ trực quan
Dashboard	Grafana	Admin monitoring
Dữ liệu	LTA DataMall API	Cung cấp nguồn dữ liệu



Hình 1.1: Minh họa các công nghệ được sử dụng

Chương 2

Định nghĩa bài toán

2.1 Mô tả bài toán thực tế được lựa chọn

Giao thông công cộng tại Singapore hàng ngày phục vụ hàng triệu lượt khách thông qua mạng lưới xe buýt, hệ thống tàu điện, đội taxi hùng hậu, hàng nghìn bãi đỗ xe và mạng lưới trạm sạc xe điện đang được mở rộng. Mỗi thành phần trong hệ thống này phát sinh dữ liệu liên tục theo thời gian thực. Từ thời gian xe đến, mức độ đông đúc của ga tàu, số chỗ đỗ xe còn trống, mật độ xe taxi rảnh,...

Bài toán được nhóm chúng em đặt ra là: làm thế nào để thu thập, lưu trữ và xử lý hiệu quả toàn bộ nguồn dữ liệu đa phương tiện này, đồng thời cung cấp giao diện tra cứu thời gian thực cho người dùng và pipeline phân tích dữ liệu lớn phục vụ nghiên cứu?

Sự khác biệt nhất giữa dự án này và các dự án dữ liệu lớn tiêu biểu chính là sự kết hợp giữa nhiều loại dữ liệu có tần suất cập nhật rất khác nhau: ETA xe buýt cập nhật mỗi 20 giây, crowd MRT cập nhật mỗi 10 phút, carpark cập nhật mỗi 1 phút, EV stations thay đổi chậm hơn. Điều này dẫn đến quyết định thiết kế quan trọng: không thể áp dụng một chiến lược polling đồng nhất cho tất cả loại data.

2.2 Đặc điểm dữ liệu và nguồn dữ liệu

API Endpoint	Dữ liệu	Cập nhật	Vai trò trong hệ thống
BusStops	5.000 trạm (tọa độ, tên)	Tĩnh	Static data, load 1 lần
BusRoutes	Các tuyến đi qua từng trạm	Tĩnh	Static data, broadcast join
v3/BusArrival?BusStopCode=X	ETA từng xe buýt	20 giây	On-demand khi user click
PCDRealTime?TrainLine=NSL	Crowd level từng ga MRT	10 phút	Kafka → Spark Streaming
CarParkAvailabilityv2	Số chỗ trống toàn Singapore	1 phút	Kafka → Spark Streaming
EVBatch	Toàn bộ trạm sạc EV (ZIP)	5 phút	Kafka → Spark Streaming
Taxi-Availability	Vị trí taxi rảnh (GeoJSON)	30 giây	Kafka → Spark Streaming

2.3 Phân tích phù hợp với Dữ liệu lớn

Bài toán giám sát giao thông công cộng đáp ứng đầy đủ các đặc trưng của Dữ liệu lớn theo mô hình 5V:

2.3.1 Volume

Mạng lưới giao thông công cộng sinh ra lượng dữ liệu rất lớn. Chỉ riêng Taxi-Availability với hàng chục nghìn điểm GPS, cập nhật mỗi 30 giây, đã tạo ra hàng triệu records mỗi ngày. Với 5.000 trạm bus, nếu poll toàn bộ mỗi 30 giây sẽ cần xử lý 10.000 request/phút, vượt quá khả năng xử lý của hệ thống truyền thống và rate limit của LTA API.

2.3.2 Velocity

Dữ liệu giao thông có velocity cao và không đồng đều theo loại: ETA xe buýt cập nhật mỗi 20 giây, carpark mỗi 1 phút, MRT crowd mỗi 10 phút. Điều này đòi hỏi hệ thống phải có khả năng tiếp nhận và xử lý dữ liệu theo thời gian thực hoặc gần thời gian thực.

2.3.3 Variety

Dữ liệu về giao thông từ API của LTA được trả về với nhiều loại định dạng: JSON thuần (bus, MRT), GeoJSON (taxi), ZIP chứa JSON (EV batch). Về kiểu dữ liệu: tọa độ địa lý (Lat/Lng), chuỗi thời gian (ETA), phân loại (crowd level: l/m/h), số nguyên (available lots). Sự đa dạng này yêu cầu hệ thống lưu trữ và xử lý linh hoạt, hỗ trợ nhiều định dạng dữ liệu khác nhau.

2.3.4 Veracity

Dữ liệu LTA có chất lượng tốt nhưng vẫn tồn tại các vấn đề: ETA đôi khi trả về giá trị âm khi xe vừa qua trạm, CrowdLevel trả về "NA" ngoài giờ hoạt động, taxi có thể có tọa độ (0, 0) lỗi. Do đó, hệ thống cần có các cơ chế làm sạch, kiểm tra và chuẩn hóa dữ liệu nhằm đảm bảo chất lượng dữ liệu đầu vào cho các bước phân tích tiếp theo.

2.3.5 Value

Giá trị của dữ liệu nằm ở khả năng hỗ trợ quyết định di chuyển theo thời gian thực cho người dùng, phát hiện pattern giờ cao điểm, xác định trạm xe buýt quan trọng trong mạng lưới giao thông và phân tích xu hướng đông đúc theo thời gian. Thông qua việc tổng hợp, phân tích và trực quan hóa dữ liệu, hệ thống có thể cung cấp các insight có giá trị phục vụ nghiên cứu, học tập và phát triển các ứng dụng giám sát giao thông thông minh trong tương lai.

2.4 Các yêu cầu nghiệp vụ

Từ bài toán thực tế, hệ thống cần đáp ứng các yêu cầu nghiệp vụ sau:

- Hiển thị bản đồ Singapore với 5.000 icon trạm bus, khi người dùng click vào trạm cần xem thì kết quả ETA real-time xuất hiện trong $<500\text{ms}$.
- Hiển thị độ đông đúc của từng ga MRT theo tuyến với màu sắc xanh/vàng/đỏ.
- Tra cứu bãi đỗ xe theo số chỗ trống còn lại, lọc theo nhà phân phối (HDB/LTA/URA).
- Tìm trạm sạc EV gần vị trí người dùng trong bán kính tùy chọn.
- Hiển thị vùng tập trung taxi rảnh trên bản đồ, cập nhật mỗi 60 giây.
- Dashboard Grafana theo dõi xu hướng ETA và đông đúc theo thời gian.

2.5 Các yêu cầu kĩ thuật

Để đáp ứng các yêu cầu nghiệp vụ, hệ thống cần thỏa mãn các yêu cầu kỹ thuật dưới đây:

- Sử dụng hạ tầng lưu trữ phân tán để đảm bảo khả năng mở rộng và độ bền dữ liệu.
- Áp dụng các công cụ xử lý Big Data như Apache Spark cho xử lý batch và streaming.
- Hỗ trợ message queue cho dữ liệu thời gian thực.
- Sử dụng cơ sở dữ liệu NoSQL để lưu trữ dữ liệu phục vụ truy vấn nhanh.
- Thiết kế hệ thống theo hướng module hóa, dễ mở rộng và tích hợp trong tương lai.

2.6 Các giả định và ràng buộc

Trong quá trình xây dựng và triển khai hệ thống, dự án được thực hiện dựa trên một số giả định và ràng buộc nhất định:

- LTA DataMall API được giả định là ổn định và trả về data hợp lệ (độ tin cậy cao thực tế).
- Hệ thống được triển khai trong môi trường học tập và nghiên cứu, chưa yêu cầu đáp ứng các tiêu chuẩn nghiêm ngặt của hệ thống giao dịch thương mại.
- Tài nguyên phần cứng và thời gian triển khai có giới hạn, do đó một số chức năng nâng cao chỉ được xây dựng ở mức độ mô phỏng hoặc định hướng.
- Dự án tập trung vào việc minh họa kiến trúc và kỹ thuật Big Data hơn là độ chính xác tuyệt đối của các mô hình phân tích.

Chương 3

Tổng quan kiến trúc hệ thống

3.1 Tổng quan kiến trúc Dữ liệu lớn

Một hệ thống Dữ liệu lớn hiện đại thường được xây dựng theo kiến trúc phân tầng, trong đó mỗi tầng đảm nhiệm một vai trò riêng biệt trong toàn bộ vòng đời dữ liệu, từ thu thập, xử lý, lưu trữ cho đến phân tích và trực quan hóa. Các tầng của hệ thống gồm:

- **Tầng thu thập (Data Ingestion):** `load_static.py` cho static data, `ingestor.py` cho streaming data.
- **Tầng xử lý (Data Processing):** Spark Structured Streaming (speed layer) và Spark Batch (batch layer).
- **Tầng lưu trữ (Data Storage):** MongoDB (static + speed + batch views), MinIO (raw Parquet + checkpoints + models).
- **Tầng phục vụ (Serving Layer):** FastAPI (10 endpoints), Web App (HTML + Leaflet.js), Grafana.

3.2 Lựa chọn mô hình kiến trúc

Đối với bài toán phân tích dữ liệu giao thông công cộng, hệ thống cần đồng thời hỗ trợ hai yêu cầu quan trọng: (1) xử lý khối lượng lớn dữ liệu lịch sử phục vụ phân tích dài hạn và (2) xử lý dữ liệu streaming để theo dõi biến động thị trường gần thời gian thực. Do đó, hai mô hình kiến trúc Big Data phổ biến là Lambda Architecture và Kappa Architecture được xem xét và so sánh.

3.2.1 Lambda Architecture

Lambda Architecture là mô hình kiến trúc Big Data kết hợp giữa xử lý Batch Layer (xử lý dữ liệu lịch sử với độ chính xác cao) và xử lý Speed Layer (xử lý real-time với độ trễ thấp), cả hai hội tụ tại Serving Layer (cung cấp kết quả cho các ứng dụng truy vấn và hiển thị).

Trong bối cảnh dự án, Lambda Architecture phù hợp với việc xử lý dữ liệu tiền điện tử khi dữ liệu lịch sử được thu thập và xử lý định kỳ thông qua các pipeline batch (Python,

Spark), trong khi dữ liệu thời gian thực được xử lý thông qua Spark Structured Streaming và Kafka để phục vụ dashboard và giám sát giao thông thông minh.

3.2.2 Kappa Architecture

Kappa Architecture sử dụng một pipeline streaming duy nhất cho cả dữ liệu lịch sử và real-time. Đơn giản hơn Lambda về mặt vận hành nhưng đòi hỏi hạ tầng streaming mạnh và phức tạp khi cần xử lý lại dữ liệu lịch sử.

Mặc dù Kappa Architecture giúp giảm độ phức tạp trong thiết kế và bảo trì hệ thống, nhưng mô hình này đòi hỏi hạ tầng streaming mạnh và phức tạp hơn, đặc biệt khi cần xử lý lại toàn bộ dữ liệu lịch sử. Trong dự án, Kappa Architecture được xem như một hướng phát triển tiềm năng trong tương lai nhưng chưa được lựa chọn làm mô hình chính.

3.2.3 So sánh và lý do lựa chọn Lambda

Lambda Architecture và Kappa Architecture đều hướng đến mục tiêu xử lý dữ liệu lớn và dữ liệu thời gian thực, tuy nhiên khác nhau ở triết lý thiết kế và cách tổ chức pipeline.

a) Ưu điểm Lambda so với Kappa cho bài toán này

Thứ nhất, Lambda Architecture cho phép tách biệt rõ ràng giữa xử lý batch và xử lý realtime, từ đó tối ưu từng pipeline theo đúng đặc thù workload. Batch Layer được thiết kế để xử lý khối lượng dữ liệu lịch sử rất lớn với mục tiêu throughput cao và độ chính xác tốt, trong khi Speed Layer tập trung vào việc xử lý dữ liệu mới phát sinh với độ trễ thấp. Ngược lại, Kappa Architecture chỉ sử dụng một pipeline streaming duy nhất cho cả dữ liệu lịch sử và realtime, dẫn đến việc cùng một cơ chế phải đồng thời đáp ứng hai yêu cầu trái ngược nhau. Khi cần replay hoặc xử lý lại khối lượng lớn dữ liệu lịch sử, pipeline streaming trong Kappa dễ bị ảnh hưởng đến hiệu năng realtime nếu hạ tầng không đủ mạnh.

Thứ hai, Lambda Architecture đơn giản hóa việc tái xử lý (reprocessing) dữ liệu lịch sử khi thay đổi logic xử lý hoặc bổ sung feature mới. Với Lambda, dữ liệu thô được lưu trữ lâu dài trong Data Lake và có thể được xử lý lại toàn bộ bằng Batch Layer để tạo ra kết quả chuẩn. Cách tiếp cận này giúp giảm rủi ro sai lệch dữ liệu và dễ kiểm soát hơn. Trong khi đó, Kappa Architecture thường phải dựa vào việc replay dữ liệu từ log streaming (ví dụ Kafka), đòi hỏi cấu hình retention phù hợp, quản lý offset phức tạp và tiềm ẩn rủi ro tắc nghẽn hoặc kéo dài thời gian xử lý khi dữ liệu lịch sử có quy mô lớn.

Thứ ba, Lambda Architecture mang lại độ ổn định và khả năng kiểm chứng kết quả tốt hơn cho các bài toán phân tích dữ liệu. Batch Layer tạo ra bộ kết quả đầy đủ và nhất quán từ dữ liệu lịch sử, đóng vai trò như “ground truth” để đối soát và kiểm thử. Speed Layer chỉ bổ sung phần dữ liệu mới nhất trong khoảng thời gian batch chưa kịp cập nhật, giúp hệ thống vừa đảm bảo tính chính xác, vừa đáp ứng yêu cầu realtime. Với Kappa Architecture, việc đảm bảo khả năng tái lập kết quả (reproducibility) và kiểm toán dữ liệu thường phụ thuộc nhiều vào trạng thái streaming và log, khiến việc kiểm chứng trở nên phức tạp hơn.

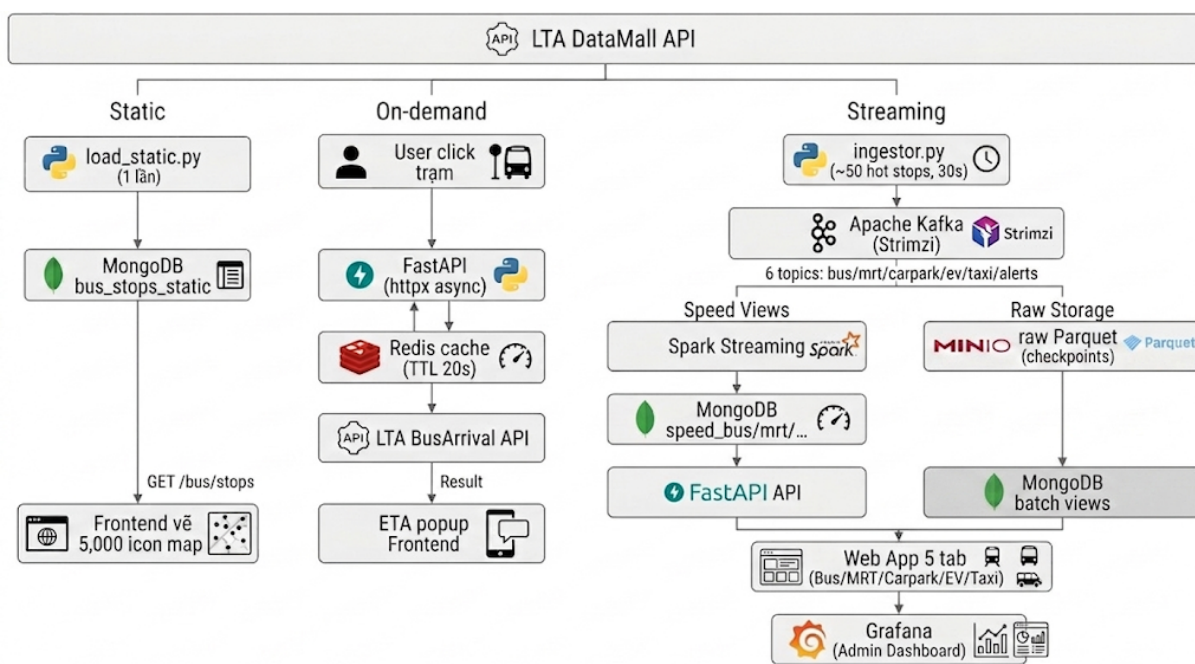
Cuối cùng, Lambda Architecture phù hợp hơn với quá trình phát triển hệ thống theo từng giai đoạn. Hệ thống có thể triển khai pipeline batch trước để nhanh chóng tạo ra dữ liệu nền và các kết quả phân tích cơ bản, sau đó từng bước bổ sung pipeline streaming khi có thêm nhu cầu về realtime. Điều này giúp giảm độ phức tạp ban đầu và phù hợp với các dự án học thuật hoặc prototype. Ngược lại, Kappa Architecture thường phù hợp

hơn với các hệ thống đã có pipeline streaming hoàn chỉnh và vận hành ổn định ngay từ đầu.

b) Sự phù hợp với bài toán

Dữ liệu giao thông có hai lớp giá trị khác nhau: giá trị thời gian thực (ETA, crowd hiện tại) và giá trị phân tích lịch sử (giờ cao điểm, pattern theo ngày trong tuần, dự đoán độ trễ). Với Speed Layer phục vụ lớp thứ nhất và Batch Layer phục vụ lớp thứ hai

Ngoài ra, On-Demand pattern là bổ sung quan trọng không có trong cả Lambda lẫn Kappa truyền thống. Đây là đặc thù của bài toán giao thông: người dùng chỉ quan tâm đến ETA khi họ thực sự click vào một trạm cụ thể, không cần pipeline liên tục cho 5.000 trạm.



Hình 3.1: Sơ đồ kiến trúc hệ thống

3.3 Luồng dữ liệu end-to-end

Luồng dữ liệu trong hệ thống được thiết kế theo hướng end-to-end, bắt đầu từ khâu thu thập dữ liệu và kết thúc ở khâu hiển thị kết quả cho người dùng. Đối với dữ liệu vị trí các trạm xe bus, vị trí các nhà ga, tuyến MRT được lưu trữ trong hệ thống lưu trữ phân tán, sau đó được xử lý batch để tổng hợp và phân tích. Đối với dữ liệu thời gian thực, dữ liệu được đẩy vào Kafka, xử lý bằng Spark Structured Streaming và ghi vào cơ sở dữ liệu phục vụ truy vấn nhanh.

Cả hai luồng dữ liệu batch và streaming đều hội tụ tại tầng lưu trữ và phục vụ, đảm bảo tính nhất quán của dữ liệu và cho phép người dùng truy cập thông tin một cách thống nhất thông qua dashboard.

Chương 4

Thiết kế chi tiết hệ thống

4.1 Thiết kế tầng Data Ingestion

4.1.1 Nguồn dữ liệu và chiến lược thu thập dữ liệu

Nguồn dữ liệu duy nhất của hệ thống là LTA DataMall API. API này được lấy bằng cách đăng ký tài khoản tại datamall.lta.gov.sg. API key được gửi về thông qua email đã đăng ký.

Để việc thu thập dữ liệu diễn ra trơn tru và không lãng phí tài nguyên, nhóm đã chia quá trình thu thập dữ liệu (ingestion) thành hai luồng xử lý riêng biệt:

- **Khởi tạo dữ liệu nền:** Với nhiệm vụ kéo toàn bộ các dữ liệu tĩnh từ LTA (như danh sách các trạm bus, các tuyến, các dịch vụ,...) file `load_static.py` chỉ chạy 1 lần duy nhất. Dữ liệu được lưu trực tiếp vào MongoDB, trong đó các thông tin trạm xe bus, bãi đỗ xe,... được đánh index tọa độ để tối ưu hóa truy vấn bản đồ. Vì đây là dữ liệu không biến động, chúng ta không cần đưa chúng qua Kafka.
- **Cập nhật dữ liệu thời gian thực:** file `ingestor.py` được chạy liên tục với nhiệm vụ quét 6 API endpoints theo các chu kỳ được cấu hình linh hoạt qua ConfigMap. Đối với dữ liệu thời gian xe bus đến, thay vì phải cập nhật toàn bộ các trạm rất tốn kém tài nguyên, hệ thống chỉ liên tục quét khoảng 50 trạm đông đúc nhất. Với các trạm còn lại, hệ thống áp dụng cơ chế lấy dữ liệu tức thời (on-demand): dữ liệu chỉ được FastAPI kéo về khi người dùng thực sự click vào trạm đó. Mọi bản ghi thu thập được ở bước này đều được đóng gói thành định dạng JSON, gắn thêm dấu thời gian và đẩy thẳng vào topic tương ứng trên Kafka.

4.1.2 Message Queue

Đối với dữ liệu thời gian thực, hệ thống sử dụng Apache Kafka làm message queue trung tâm cho tầng streaming ingestion. Kafka được triển khai bằng Strimzi trên môi trường Kubernetes thông qua các manifest cấu hình nhằm đảm bảo khả năng mở rộng và tính sẵn sàng cao.

Topic	Partitions	Interval
bus-arrivals	3	30s
mrt-crowd	2	10 phút
carpark-lots	2	60s
ev-stations	2	5 phút
taxi-positions	2	30s
train-alerts	1	5 phút

Bảng 4.1: Cấu hình Kafka Topics

4.1.3 Chiến lược xử lý dữ liệu đến trễ và trùng lặp

Trong môi trường xử lý luồng (streaming), việc dữ liệu đến muộn hoặc bị gửi trùng lặp là vấn đề thường xuyên xảy ra. Để đảm bảo tính chính xác và hiệu năng, hệ thống áp dụng một chiến lược kiểm soát gồm ba lớp:

- **Quản lý dữ liệu đến trễ:** Ngay từ khâu thu thập, mỗi tin nhắn gửi vào Kafka đều được gắn sẵn nhãn thời gian sự kiện. Dựa vào nhãn này, Spark Streaming thiết lập một ngưỡng thời gian chờ là 3 phút. Bất kỳ bản ghi nào đến trễ quá ngưỡng này sẽ tự động bị loại bỏ, giúp giải phóng bộ nhớ trạng thái (state store) và ngăn hệ thống bị quá tải bởi các dữ liệu đã hết giá trị.
- **Loại bỏ dữ liệu trùng lặp:** Để tránh tình trạng nhận nhiều lần cùng một thông tin thì script `ingestor.py` sử dụng khóa kết hợp định dạng `stop_code - service_no`. Cách làm này đã đảm bảo mọi tin nhắn của cùng một xe bus sẽ luôn được điều hướng vào cùng một partition trong Kafka. Trước khi ghi xuống MongoDB, Spark Streaming sẽ dùng hàm `dropDuplicates` để rà soát và loại bỏ các bản ghi trùng nhau dựa trên ba yếu tố cốt lõi: mã trạm, số tuyến và thời gian dự kiến đến.
- **Ghi đè an toàn:** Để nhằm rủi ro hệ thống bị gián đoạn và khởi động lại, các truy vấn tốc độ cao trong MongoDB được thiết kế theo cơ chế `upsert`. Hệ thống tạo ra một khóa chính duy nhất kết hợp từ mã trạm, số tuyến và thời gian bắt đầu của cửa sổ tính toán (`window_start`). Nhờ vậy mà, dù toàn bộ pipeline có phải chạy lại thì dữ liệu lưu trữ vẫn được đảm bảo không bao giờ bị nhân bản sai lệch.

4.2 Thiết kế tầng Data Processing với Apache Spark

Tại tầng xử lý dữ liệu, hệ thống sử dụng framework phân tán Apache Spark làm nền tảng cốt lõi để giải quyết đồng thời hai bài toán: phân tích dữ liệu lịch sử (Batch Processing) và xử lý dữ liệu thời gian thực (Stream Processing).

4.2.1 Batch Processing (Xử lý dữ liệu định kỳ)

Nhằm tối ưu hóa tài nguyên, hệ thống mà nhóm xây dựng đã tận dụng cơ chế lập lịch của Kubernetes thông qua đối tượng `ScheduledSparkApplication`. Theo cấu hình, tiến trình Spark Batch sẽ được kích hoạt tự động vào lúc 2 giờ sáng mỗi ngày để thực hiện các luồng xử lý dữ liệu quy mô lớn.

Quá trình này bắt đầu bằng việc thu thập dữ liệu Parquet thô từ MinIO. Pipeline batch đọc raw Parquet từ MinIO từ địa chỉ `s3a://sg-transit-data/raw/bus/`. Nhờ áp dụng cơ chế loại bỏ phân vùng (partition pruning), Spark chỉ truy xuất tập dữ liệu thuộc 30 ngày gần nhất nhằm tối ưu hiệu suất đọc. Dữ liệu sau đó trải qua các phép biến đổi thông qua các hàm cửa sổ thời gian (window functions) và các hàm tự định nghĩa (UDF), kết hợp thông tin thông qua kỹ thuật broadcast join, và thực hiện tái cấu trúc dữ liệu (pivot) để tạo ra các báo cáo tổng hợp. Cuối cùng, tập dữ liệu đã qua xử lý được sử dụng để huấn luyện mô hình học máy với Spark MLlib và phân tích đồ thị mạng lưới giao thông với thuật toán PageRank của GraphFrames. Toàn bộ kết quả đầu ra được lưu trữ vào MongoDB để phục vụ tăng truy vấn.

4.2.2 Stream Processing (Xử lý dữ liệu luồng)

Khác với Batch Processing chịu trách nhiệm tổng hợp dữ liệu định kỳ, Stream Processing đảm nhận vai trò xử lý luồng dữ liệu thời gian thực. Một ứng dụng Spark Structured Streaming duy nhất được triển khai để tiêu thụ (consume) dữ liệu đồng thời từ 5 topic của Kafka. Ưu điểm của thiết kế này là mỗi luồng xử lý đều duy trì một cơ chế kiểm tra trạng thái (checkpoint) độc lập trên MinIO, giúp đảm bảo tính cô lập và khả năng phục hồi lỗi của từng luồng mà không ảnh hưởng đến toàn bộ tiến trình.

Đặc biệt đối với luồng dữ liệu xe buýt, hệ thống đã áp dụng cơ chế xử lý phân nhánh:

- **Luồng tốc độ cao:** Dữ liệu được tính toán và tổng hợp theo từng cửa sổ thời gian rồi sau đó được cập nhật trực tiếp lên MongoDB để đáp ứng yêu cầu hiển thị thời gian thực trên bảng điều khiển (dashboard).
- **Luồng lưu trữ thô:** Dữ liệu thô đồng thời được sao lưu trực tiếp xuống kho lưu trữ MinIO dưới định dạng Parquet, đóng vai trò là nguồn dữ liệu gốc phục vụ cho quá trình xử lý Batch định kỳ.

4.2.3 Windowing & Watermarking (Cửa sổ thời gian và Lọc trễ)

```
1 # Cửa sổ trượt (Sliding Window): Gom cum du lieu moi 5 phut, tan
   suat cap nhat 1 phut
2 bus_agg = bus_watermarked \
3     .groupBy(
4         F.window("ingested_at", "5 minutes", "1 minute"),
5         "bus_stop_code", "service_no"
6     ) \
7     .agg(
8         F.avg("eta_seconds").alias("avg_eta_seconds"),
9         F.min("eta_seconds").alias("min_eta_seconds"),
10        F.last("load_level").alias("current_load")
11    )
```

```
12  
13 # Watermark 3 phút: Ngưỡng dung sai tối đa cho dữ liệu đến trễ  
14 bus_watermarked = bus_parsed.withWatermark("ingested_at", "3  
    minutes")
```

Listing 4.1: Cấu hình thuật toán gom cụm dữ liệu luồng xe buýt

Việc đưa ra thiết lập Watermark là 3 phút được căn cứ trên đặc thù dữ liệu thực tế: API của LTA có chu kỳ cập nhật khoảng 20 giây, kết hợp với độ trễ tối đa của mạng lưới ước tính khoảng 60 giây. Khoảng dung sai 3 phút này được đánh giá là một ngưỡng an toàn, vừa đảm bảo khả năng tiếp nhận các thông điệp đến muộn do độ trễ đường truyền, vừa hạn chế tình trạng quá tải bộ nhớ khi hệ thống phải duy trì trạng thái của các bản ghi đã quá hạn.

4.2.4 Quản lý trạng thái

Nhằm tăng khả năng chịu lỗi của ứng dụng Spark trong các trường hợp xảy ra sự cố gián đoạn hoặc khởi động lại máy chủ, toàn bộ trạng thái tính toán được lưu trữ và quản lý thông qua cơ sở dữ liệu trạng thái và đồng bộ lên MinIO. Tại mỗi chu kỳ micro-batch (30 giây), hệ thống chỉ thực hiện xác nhận (commit) trạng thái lưu trữ sau khi dữ liệu đã được ghi nhận thành công vào MongoDB. Cơ chế này bảo đảm rằng khi hệ thống khôi phục, Spark có thể đọc lại trạng thái từ điểm kiểm tra (checkpoint) gần nhất và tiếp tục quá trình tiêu thụ dữ liệu từ vị trí tương ứng (offset) trên Kafka một cách liên mạch.

4.2.5 Cơ chế Exactly-Once

Nhằm đảm bảo tính toàn vẹn của dữ liệu, hệ thống thực thi ngăn chặn hoàn toàn các sự cố thiếu sót hay lặp lại dữ liệu thông qua ba cơ chế phối hợp:

- **Quản lý Offset nghiêm ngặt:** Spark duy trì và theo dõi vị trí đọc trên Kafka thông qua các checkpoint, đảm bảo tính liên tục của luồng xử lý.
- **Ghi đè dựa trên khóa định danh:** Với MongoDB, mỗi bản ghi được cấp phát một khóa chính (_id) duy nhất kết hợp từ các định danh nghiệp vụ. Khóa này đảm bảo rằng các thao tác ghi bị lặp lại (ví dụ do chạy lại pipeline) sẽ chỉ thực hiện cập nhật (upsert) bản ghi thay vì tạo ra dữ liệu dư thừa.
- **Đảm bảo tính nguyên tử khi ghi Parquet:** Khi lưu trữ dữ liệu vào MinIO, Spark sẽ thao tác trên các tệp tạm thời. Lệnh đổi tên tệp (atomic rename) thành tệp chính thức chỉ được kích hoạt khi quá trình ghi hoàn tất 100%, từ đó ngăn chặn triệt để sự xuất hiện của các tệp dữ liệu hỏng (corrupted files) do mất kết nối đột ngột.

4.3 Thiết kế tầng Data Storage

4.3.1 MinIO - Object Storage

Để giải quyết bài toán lưu trữ lớn mà không phải phụ thuộc vào các dịch vụ đám mây đắt đỏ như Google Cloud Storage (GCS) hay Amazon S3, nhóm đã quyết định triển khai MinIO làm kho lưu trữ đối tượng (Object Storage) ngay trên cụm Kubernetes nội bộ.

MinIO đóng vai trò như một "kho chứa dữ liệu thô" khổng lồ, chạy dưới dạng một Deployment trong namespace data và được tiếp sức bởi một ổ đĩa ảo (PVC) dung lượng 20Gi. Với API endpoint nội bộ là `http://minio.data.svc.cluster.local:9000`, Apache Spark có thể dễ dàng đọc ghi dữ liệu thông qua connector `hadoop-aws` với tiền tố `s3a://` quen thuộc. Nhóm cũng cấu hình `fs.s3a.path.style.access=true` để đảm bảo dữ liệu di chuyển trơn tru không gặp rào cản.

Bên trong bucket chính `sg-transit-data`, dữ liệu được tổ chức một cách ngăn nắp và khoa học:

```
sg-transit-data/  
├── raw/bus/  
│   └── Nơi Spark Streaming ghi lưu trữ thô Parquet (giữ lại 3  
│       tiếng).  
├── raw/mrt/  
│   └── Tương tự cho dữ liệu tàu điện MRT  
├── raw/carpark/  
│   └── Tương tự cho dữ liệu bãi đỗ xe  
├── raw/taxi/  
│   └── Tương tự cho dữ liệu định vị taxi  
├── checkpoints/  
│   └── Sổ tay ghi nhớ vị trí (checkpoint) của Spark Streaming  
│       (không xóa).  
└── jobs/  
    └── Nơi lưu trữ mã nguồn các job PySpark (streaming_job.py,  
        batch_job.py).
```

Để ổ đĩa 20Gi không bị quá tải bởi dòng chảy dữ liệu streaming đổ về liên tục, một CronJob dọn dẹp tự động được cài đặt để quét dọn mỗi giờ một lần, xóa các thư mục dữ liệu thô (`raw/*`) đã cũ hơn 3 tiếng, giữ cho dung lượng ổ đĩa luôn ở mức an toàn.

4.3.2 NoSQL Database

MongoDB được sử dụng làm Serving Store cho hệ thống. Để đảm bảo dữ liệu không bị biến mất khi các container bị khởi động lại, MongoDB được thiết lập dưới dạng một StatefulSet với PVC 10Gi.

Các bảng dữ liệu (Collection) được phân chia rõ ràng theo từng nhiệm vụ cụ thể:

Collection	Phân loại	Chiến lược dọn dẹp	Vai trò chi tiết
bus_stops_static	Dữ liệu tĩnh	Không bao giờ xóa	Lưu thông tin cố định của 5.000 trạm xe buýt để vẽ bản đồ
bus_routes_static	Dữ liệu tĩnh	Không bao giờ xóa	Lưu lộ trình chi tiết phục vụ các truy vấn liên quan
speed_bus/mrt/carpark/ev/taxi	Speed views	Tự động dọn dẹp dữ liệu > 3h	Chứa dữ liệu thời gian thực cập nhật liên tục từ Spark Streaming
batch_hourly_pivot	Batch views	Lưu trữ trong vòng 7 ngày	Báo cáo tổng hợp số liệu theo giờ phục vụ phân tích
batch_bus_daily	Batch views	Lưu trữ trong vòng 7 ngày	Thống kê hiệu suất hoạt động của các tuyến buýt theo ngày
batch_graph_pagerank	Batch views	Lưu trữ trong vòng 7 ngày	Kết quả phân tích độ quan trọng của các trạm xe buýt từ GraphFrames

Bảng 4.2: Các collection và vai trò trong hệ thống dữ liệu

4.3.3 Cache Layer

Để hỗ trợ MongoDB và ngăn chặn việc gọi API dồn dập lên hệ thống LTA DataMall (tránh tình trạng bị khóa tài khoản do vượt rate limit), nhóm đã đưa thêm Redis đóng vai trò bộ nhớ đệm (Cache Layer). Chạy dưới dạng StatefulSet với PVC 2Gi, Redis giải quyết xuất sắc hai bài toán:

- **Bộ đếm thời gian xe đến:** Khi người dùng nhấp vào một trạm xe buýt bất kỳ, thông tin ETA sẽ được lưu tạm vào Redis với mã khóa `bus:arrivals:stop_code` trong vòng 20 giây. Nếu có rất nhiều người cùng xem một trạm trong khoảng thời gian này, hệ thống chỉ gọi API đúng 1 lần duy nhất, giải phóng áp lực lên cả backend và API.
- **Bộ danh sách trạm:** Danh sách hơn 5000 trạm xe bus là lượng dữ liệu lớn nhưng ít thay đổi. Thay vì bắt MongoDB phải truy vấn liên tục mỗi khi có người dùng truy cập web, danh sách này được nén và lưu vào Redis với khóa `bus:stops:all` có thời gian 1 tiếng.

4.3.4 Chiến lược phân vùng và nén

Dữ liệu đổ về liên tục như "thác lũ" đòi hỏi một chiến lược tổ chức thông minh để tránh hiện tượng quá tải khi truy vấn dữ liệu lịch sử.

Khi ghi dữ liệu xuống MinIO, Spark sẽ tự động phân chia (partition) dữ liệu xe buýt theo giờ Singapore (partitionBy("hour_sg")). Nhờ cấu trúc thư mục dạng cây này, khi Batch Layer cần tính toán, nó có thể bỏ qua hàng triệu dòng dữ liệu không liên quan và chỉ nhắm thẳng vào phân vùng giờ mong muốn (cơ chế partition pruning), giúp đẩy nhanh tốc độ xử lý lên gấp nhiều lần.

Đồng thời, định dạng Parquet được kết hợp với thuật toán nén Snappy mặc định của Spark. Sự kết hợp này mang lại hiệu quả kép: vừa tiết kiệm đáng kể không gian lưu trữ trên MinIO, vừa duy trì tốc độ giải nén cực nhanh trong quá trình đọc ghi.

Về phía MongoDB, nhóm chủ động không sử dụng cơ chế TTL index tự động của cơ sở dữ liệu. Thay vào đó, việc dọn dẹp các bản ghi cũ được giao trọn cho một Kubernetes CronJob chạy độc lập bên ngoài. Lựa chọn này giúp nhóm chủ động kiểm soát thời điểm và khối lượng dữ liệu bị xóa, tránh gây đột biến tải (I/O spikes) cho hệ thống trong giờ cao điểm.

4.4 Thiết kế tầng Analytics & Visualization

Để người dùng có thể tương tác mượt mà với lượng dữ liệu khổng lồ, hệ thống sử dụng kiến trúc tách biệt giữa Backend phục vụ API và Frontend hiển thị:

- **FastAPI Backend:** Nhóm chọn FastAPI nhờ khả năng xử lý bất đồng bộ (async) cực kỳ xuất sắc. Thay vì dùng các thư viện đồng bộ thông thường, hệ thống sử dụng `httpx AsyncClient` để thực hiện các cuộc gọi API on-demand đến LTA mà không chặn luồng (non-blocking). Backend cung cấp 10 API endpoints, được phân chia gọn gàng thành 5 khu vực chức năng (Xe buýt, MRT, Bãi đỗ xe, Trạm sạc EV, và Taxi).
- **Web App Frontend:** Giao diện được xây dựng thuần túy bằng HTML, JavaScript và thư viện bản đồ tương tác Leaflet.js, mang trên mình giao diện tối màu (Dark Theme) hiện đại. Giao diện gồm 5 tab tính năng nhằm thuận tiện cho người sử dụng.
- **Grafana Dashboard:** Nhằm mục đích theo dõi biến động giao thông tổng thể, nhóm thiết lập một bảng điều khiển Grafana liên kết trực tiếp với cơ sở dữ liệu MongoDB. Bảng điều khiển này tự động làm mới mỗi 30 giây, cung cấp 5 biểu đồ trực quan sinh động như: biểu đồ chuỗi thời gian của ETA xe buýt, bản đồ nhiệt (heatmap) mức độ đông đúc tại các ga MRT, biểu đồ cột thể hiện sức chứa bãi đỗ xe, hay mật độ taxi đang hoạt động trên toàn đảo.

4.5 Thiết kế tầng Monitoring & Logging

Vận hành một hệ thống Big Data gồm nhiều thành phần trên môi trường Minikube giới hạn tài nguyên (16GB RAM) đòi hỏi một chiến lược giám sát và quản lý tài nguyên cực kỳ khắt khe:

- **Giám sát đa tầng:** Tình trạng của hệ thống được theo dõi ở nhiều cấp độ khác nhau. Quản trị viên có thể xem chi tiết quá trình qua `kubectl logs`, giám sát các tiến trình xử lý nặng của Spark qua giao diện Spark UI (thông qua cơ chế port-forward), theo dõi luồng dữ liệu thời gian thực trên Grafana, và liên tục kiểm tra điểm cuối `/health` của FastAPI để đảm bảo các dịch vụ vẫn đang hoạt động ổn định.
- **Quản lý vết tích:** Để tránh việc log của các tác vụ tự động làm đầy ổ cứng ảo, các tiến trình dọn dẹp (CronJob) được cấu hình khéo léo với tham số `successful`

`JobsHistoryLimit`: 3 giúp hệ thống tự động dọn dẹp và chỉ giữ lại vết tích của 3 lần chạy thành công gần nhất.

- **Giới hạn tài nguyên:** Đây là yếu tố sống còn. Nhóm đã cẩn thận thiết lập các mức giới hạn (limits/requests) riêng biệt cho từng container (từ Kafka, Spark đến MongoDB). Thiết lập này đóng vai trò như một "cầu chì an toàn", ngăn chặn triệt để tình trạng rò rỉ bộ nhớ khiến một thành phần tiêu thụ cạn kiệt tài nguyên của Minikube, dẫn đến việc bị hệ điều hành tiêu diệt đột ngột (hiện tượng OOM kill).

Chương 5

Triển khai hệ thống

Hệ thống được thiết kế và triển khai trên môi trường giả lập cụm Kubernetes cục bộ thông qua Minikube (hệ điều hành Windows, dung lượng RAM 16GB, sử dụng Docker Desktop). Lựa chọn kiến trúc này đáp ứng được yêu cầu về việc mô phỏng một môi trường phân tán thực tế nhưng vẫn tối ưu hóa được chi phí duy trì dịch vụ điện toán đám mây, đồng thời đặt ra các yêu cầu về kỹ thuật quản lý tài nguyên nghiêm ngặt.

5.1 Kiến trúc môi trường và Phân bổ tài nguyên

Việc triển khai các thành phần hạ tầng dữ liệu lớn (Kafka, Spark, MongoDB, MinIO) trên cấu hình RAM 16GB đòi hỏi một chiến lược phân bổ tài nguyên chặt chẽ:

- **Tác vụ xử lý:** Được cấp phát xấp xỉ 4GB RAM (bao gồm bộ điều phối Driver và các tác vụ thực thi Executors) để đảm bảo năng lực tính toán cho quá trình chuyển đổi và tổng hợp dữ liệu.
- **Hệ thống Message Queue:** Được cấp phát 1.5GB RAM nhằm duy trì sự ổn định của luồng dữ liệu thời gian thực.
- **Hệ thống lưu trữ và cơ sở dữ liệu:** MinIO, MongoDB, Redis và các dịch vụ phụ trợ sử dụng chung khoảng 3GB RAM. Khoảng 2GB dung lượng còn lại được dành cho các tác vụ nền của hệ điều hành nhằm tránh hiện tượng quá tải hệ thống.

Môi trường Minikube được phân tách logic thành các Namespaces chuyên biệt nhằm tăng cường khả năng quản lý:

- (1) **kafka:** Chứa các thành phần của cụm Kafka (chế độ KRaft) và Strimzi Operator.
- (2) **data:** Chứa cụm cơ sở dữ liệu (MongoDB, Redis) và không gian lưu trữ đối tượng (MinIO).
- (3) **transit:** Chứa các ứng dụng thu thập và phục vụ dữ liệu (Ingestor, FastAPI) cùng các tác vụ định kỳ.
- (4) **spark-operator** và **monitoring:** Chứa các công cụ điều phối tác vụ và giám sát hệ thống (Grafana).

Đối với không gian lưu trữ, Storage class standard của Minikube được sử dụng để tự động cấp phát tài nguyên lưu trữ liên tục (PVC): MinIO được cấp 20Gi, Kafka 10Gi, MongoDB 10Gi và Redis 2Gi.

5.2 Chiến lược và quy trình triển khai

Quy trình triển khai của hệ thống được thực hiện theo ba phân luồng độc lập: Luồng xử lý thời gian thực (Streaming path), Luồng xử lý theo lô (Batch path) và Luồng dọn dẹp hệ thống (Cleanup path). Toàn bộ quá trình được mã hóa thành các kịch bản triển khai tự động (declarative manifests):

1. **Khởi tạo môi trường:** Kích hoạt cụm Minikube với các giới hạn tài nguyên được thiết lập trước (`minikube start -memory=10240 -cpus=4`).
2. **Triển khai hạ tầng nền tảng:** Áp dụng các cấu hình YAML để triển khai Kafka Cluster cùng 6 topics giao tiếp; sau đó khởi chạy MongoDB, Redis và MinIO.
3. **Thiết lập không gian lưu trữ:** Khởi tạo các không gian chứa dữ liệu (buckets) trong MinIO phục vụ việc lưu trữ dữ liệu thô, trạng thái hệ thống (checkpoints), và mô hình học máy.
4. **Triển khai ứng dụng phân tích:** Tải các tệp mã nguồn xử lý dữ liệu (`streaming_job.py`, `batch_job.py`) lên MinIO.
5. **Kích hoạt luồng dữ liệu:** Khởi chạy module Ingestor để bắt đầu quá trình thu thập dữ liệu; triển khai FastAPI và Grafana phục vụ giao diện người dùng và quản trị viên.
6. **Khởi động tiến trình xử lý:** Kích hoạt tác vụ Spark Streaming để xử lý dữ liệu liên tục; cấu hình tác vụ Spark Batch hoạt động tự động vào lúc 2 giờ sáng hàng ngày; và thiết lập CronJob dọn dẹp dữ liệu cũ theo chu kỳ hàng giờ.

5.3 Cấu hình và Bảo mật hệ thống

Tính linh hoạt và an toàn của hệ thống được đảm bảo thông qua các cơ chế quản lý tích hợp của Kubernetes:

- **Quản lý cấu hình (ConfigMap):** Các tham số môi trường và tham số điều khiển của hệ thống (ví dụ: chuỗi kết nối cơ sở dữ liệu, tần suất cập nhật dữ liệu) được trừu tượng hóa vào đối tượng `transit-config`. Phương pháp này cho phép thay đổi cấu hình vận hành mà không làm gián đoạn quá trình hoạt động của các container hoặc yêu cầu biên dịch lại mã nguồn.
- **Quản lý dữ liệu nhạy cảm (Secrets):** Các thông định danh và xác thực (API Key, mật khẩu truy cập cơ sở dữ liệu) được mã hóa và lưu trữ trong đối tượng Kubernetes Secret. Các pod chỉ được cấp quyền truy cập mức độ đọc (read-only mount) đối với các thông tin này. Ngoài ra, các tệp tin chứa thông tin nhạy cảm được loại bỏ khỏi phiên bản mã nguồn quản lý thông qua cấu hình `.gitignore` nghiêm ngặt.
- **Chính sách mạng (NetworkPolicy):** Việc giao tiếp bên trong cụm được kiểm soát chặt chẽ nhằm giảm thiểu rủi ro bảo mật. Namespace `transit` là khu vực duy nhất được cấp quyền gọi dữ liệu ra môi trường mạng bên ngoài (Internet) để thực hiện tác vụ thu thập qua API. MinIO được cách ly hoàn toàn, chỉ cho phép kết nối trong phạm vi cụm nội bộ (ClusterIP). Các cổng giao tiếp cho người dùng cuối (FastAPI, Grafana) được bộc lộ một cách có kiểm soát thông qua cơ chế NodePort của Kubernetes.

Chương 6

Xử lý dữ liệu với Apache Spark

Chương này trình bày chi tiết cách hệ thống sử dụng Apache Spark để xử lý dữ liệu tiền điện tử ở cả hai chế độ batch và streaming. Nội dung tập trung mô tả các quy trình xử lý dữ liệu, các bước biến đổi chính, các phép tổng hợp và tối ưu hiệu năng được áp dụng.

6.1 Kiến trúc luồng xử lý dữ liệu

Pipeline xử lý dữ liệu của hệ thống được thiết kế theo kiến trúc Lambda, bao gồm hai luồng xử lý bổ sung cho nhau:

- **Pipeline streaming:** Được triển khai thông qua module `streaming_job.py`, luồng này bao gồm 5 truy vấn Spark Structured Streaming vận hành song song. Mỗi truy vấn chịu trách nhiệm tiêu thụ dữ liệu từ một Kafka topic cụ thể. Kết quả đầu ra được định tuyến đồng thời đến cơ sở dữ liệu MongoDB (dưới dạng các bảng dữ liệu tổng hợp tức thời - speed views) và kho lưu trữ MinIO (dưới dạng dữ liệu thô định dạng Parquet).
- **Pipeline batch:** Được thực thi định kỳ thông qua module `batch_job.py`. Tác vụ Spark Batch truy xuất dữ liệu thô từ MinIO, thực hiện một chuỗi 8 bước chuyển đổi và tính toán phức tạp. Đầu ra của quá trình này bao gồm các bảng dữ liệu tổng hợp (batch views) được lưu trữ vào MongoDB và các mô hình học máy (MLlib model) phục vụ dự đoán.

Hai pipeline này được vận hành song song và thống nhất về schema dữ liệu, đảm bảo tính nhất quán giữa dữ liệu realtime và dữ liệu lịch sử.

6.2 Kỹ thuật chuyển đổi dữ liệu

6.2.1 Chuỗi chuyển đổi đa giai đoạn

Các pipeline Spark được thiết kế theo nhiều giai đoạn xử lý liên tiếp, mỗi giai đoạn đảm nhiệm một nhiệm vụ rõ ràng trong quy trình xử lý dữ liệu. Quy trình chung bao gồm:

- **Phân giải:** Chuyển đổi chuỗi JSON thô thành đối tượng DataFrame có cấu trúc (typed DataFrame) thông qua hàm `from_json`.
- **Chuẩn hóa:** Thực hiện ép kiểu dữ liệu (casting) cho các trường thời gian, lọc bỏ các bản ghi không hợp lệ (null values) và định dạng lại tên cột.
- **Làm giàu dữ liệu:** Áp dụng các hàm tự định nghĩa (UDF) để giải mã các giá trị mã hóa từ API nguồn và tính toán các trường dữ liệu phái sinh (ví dụ: `eta_seconds`, `hour_sg`, `is_peak_hour`).
- **Tổng hợp:** Áp dụng các kỹ thuật gom nhóm (`groupBy`) kết hợp với cửa sổ thời gian (window functions) và cơ chế kiểm soát dữ liệu trễ (watermark).
- **Lưu trữ:** Xuất kết quả cuối cùng đến các hệ thống lưu trữ đích (MongoDB và MinIO).

6.2.2 Tối ưu hóa chuỗi thực thi

Các phép biến đổi dữ liệu được thiết kế dưới dạng chuỗi liên kết (chaining operations). Nhờ cơ chế Catalyst Optimizer của Spark, các phép biến đổi tuần tự này được tự động gộp (folding) thành một kế hoạch thực thi (execution plan) tối ưu duy nhất, loại bỏ các bước tính toán trung gian không cần thiết, từ đó tối đa hóa hiệu suất phần cứng.

6.2.3 Các hàm tự định nghĩa

Trong phạm vi dự án, các pipeline Spark ưu tiên sử dụng các hàm built-in của Spark để thực hiện biến đổi và tính toán. Việc hạn chế sử dụng hàm tùy chỉnh (UDF) giúp đảm bảo khả năng tối ưu của Spark, đặc biệt trong các pipeline streaming yêu cầu độ trễ thấp. Các hàm tùy chỉnh chỉ được xem xét khi công thức tính toán không thể biểu diễn bằng các hàm sẵn có, và trong trường hợp đó cần được kiểm thử kỹ lưỡng để tránh ảnh hưởng đến hiệu năng và tính ổn định của hệ thống.

6.3 Kỹ thuật tổng hợp dữ liệu

Trong Tầng Xử lý Theo lô (Batch Layer), các hàm cửa sổ (Window functions) được ứng dụng mạnh mẽ để tính toán các tham số thống kê trượt trên chuỗi thời gian lịch sử. Cụ thể, hệ thống định nghĩa các cửa sổ phân vùng theo mã trạm (`bus_stop_code`) và mã tuyến (`service_no`), kết hợp sắp xếp theo thời gian (`ingested_at`) nhằm trích xuất giá trị trung bình trượt (rolling average) và độ lệch chuẩn (standard deviation) trong các khung thời gian 1 giờ và 24 giờ. *Đoạn mã minh họa cấu trúc Window Function trong Spark:*

```
1 window_1h = Window \  
2     .partitionBy("bus_stop_code", "service_no") \  
3     .orderBy("ingested_at") \  
4     .rowsBetween(-11, 0) # Khung 1 giờ (12 bản ghi x 5 phút)  
5  
6 df_windowed = df_raw \  
7     .withColumn("rolling_avg_1h", F.avg("eta_seconds").over(  
     window_1h))
```

6.4 Tối ưu hóa các phép kết nối

6.4.1 Phép kết nối quảng bá

Nhằm tối ưu hóa hiệu suất khi thực hiện phép kết nối giữa tập dữ liệu giao dịch khổng lồ (hàng triệu bản ghi trạng thái xe buýt) với tập dữ liệu từ điển (metadata của 5.000 trạm xe buýt), hệ thống áp dụng chiến lược Broadcast Join. Thay vì phải thực hiện trộn dữ liệu trên toàn cụm (shuffle) rất tốn kém tài nguyên, bảng siêu dữ liệu trạm (bus_meta) được sao chép và phân phối trực tiếp (broadcast) đến bộ nhớ của tất cả các Executor, giảm thiểu tối đa chi phí truyền tải mạng và tăng tốc độ xử lý.

Đoạn mã minh họa kỹ thuật Broadcast Join và Caching:

```
1 bus_meta.cache() # Lưu vào bộ nhớ trước khi phân phối
2
3 df_final = df_enriched.join(
4     F.broadcast(bus_meta),
5     df_enriched["bus_stop_code"] == bus_meta["BusStopCode"],
6     "left"
7 )
```

6.4.2 Phép kết nối cấu trúc đồ thị

Trong bài toán phân tích mạng lưới tuyến đường, kỹ thuật kết nối sử dụng hàm trượt (lag function) được vận dụng để thiết lập các cạnh (edges) của đồ thị. Kỹ thuật này giúp hệ thống xác định chính xác trình tự của các trạm xe buýt kề nhau (stop sequence) trên cùng một tuyến đường, tạo tiền đề dữ liệu cho thuật toán PageRank trong thư viện GraphFrames.

6.5 Các kỹ thuật tối ưu hóa hiệu năng

Để đảm bảo hệ thống vận hành trơn tru trong giới hạn tài nguyên cấp phát, hàng loạt các chiến lược tối ưu hóa đã được thực thi:

- **Tối ưu hóa quá trình quét:** Quá trình lưu trữ dữ liệu thô trên MinIO được cấu trúc phân vùng theo giờ (hour_sg). Khi Tầng Batch thực hiện truy xuất dữ liệu của 30 ngày gần nhất, trình tối ưu hóa của Spark sẽ tự động bỏ qua (prune) các thư mục phân vùng nằm ngoài khoảng thời gian này, cắt giảm đáng kể chi phí đọc ổ đĩa cứng (I/O).
- **Phân nhóm dữ liệu:** Bảng dữ liệu lịch sử tổng hợp được phân chia cố định thành 8 buckets dựa trên trường service_no và sắp xếp cục bộ theo thời gian. Kỹ thuật này gia tăng đáng kể tốc độ cho các truy vấn phân tích cần thực hiện nhóm (aggregation) hoặc kết nối theo tuyến xe buýt trong tương lai.
- **Chiến lược Bộ nhớ đệm:** Bảng dữ liệu tính bus_meta được nạp vào bộ nhớ (cache) trước khi tham gia vào các phép toán Broadcast Join và cấu trúc GraphFrames, qua đó triệt tiêu việc phải truy vấn lại cơ sở dữ liệu MongoDB nhiều lần.

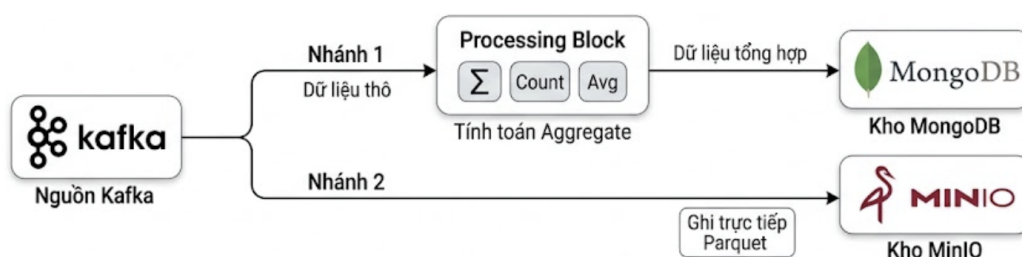
- **Phân tích kế hoạch thực thi:** Kế hoạch thực thi của toàn bộ quá trình được kiểm tra (sử dụng phương thức `explain(mode="extended")`) để đảm bảo rằng Spark đã lựa chọn `BroadcastHashJoin` thay vì `SortMergeJoin`, đồng thời xác nhận rằng các bộ lọc phân vùng (`PartitionFilters`) đã được áp dụng hiệu quả ở mức đọc file Parquet.

Chương 7

Xử lý dữ liệu theo thời gian thực

Chương này trình bày chi tiết quy trình xử lý dữ liệu streaming trong hệ thống, dựa trên Apache Spark Structured Streaming. Nội dung tập trung làm rõ kiến trúc pipeline streaming, cách thức ghi dữ liệu ra các hệ thống lưu trữ khác nhau, cơ chế xử lý dữ liệu đến trễ, quản lý trạng thái và đảm bảo tính nhất quán của dữ liệu trong môi trường xử lý thời gian thực.

7.1 Kiến trúc Structured Streaming



Hình 7.1: Sơ đồ luồng dữ liệu kép (dual-branch flow) của streaming pipeline

Luồng xử lý thời gian thực được thiết kế dựa trên mô hình Micro-batch với chu kỳ kích hoạt là 30 giây. Mỗi khi chu kỳ điểm nhịp, hệ thống chủ động kéo hàng đợi tin nhắn từ Kafka, đi qua bộ chuyển đổi DataFrame và phân luồng đầu ra tới hai đích đến hoàn toàn khác biệt:

- **Luồng tổng hợp Tốc độ cao:** Dữ liệu được tính toán tức thời (ví dụ: gom nhóm theo 5 phút) và ghi đè vào cơ sở dữ liệu NoSQL để phục vụ giao diện người dùng ngay lập tức.
- **Luồng lưu trữ Dữ liệu thô:** Song song với quá trình trên, luồng dữ liệu gốc được đẩy thẳng vào không gian lưu trữ phân tán MinIO dưới dạng file Parquet, bảo toàn nguyên vẹn mọi thông tin cho Tầng Batch xử lý sau này.

Để tối đa hóa tài nguyên trên môi trường Minikube giới hạn, 5 luồng truy vấn tương ứng với 5 loại phương tiện được điều phối chạy song song bên trong cùng một ngữ cảnh, chia sẻ chung tài nguyên tính toán nhưng duy trì các điểm neo phục hồi độc lập để ngăn ngừa rủi ro sụp đổ dây chuyền.

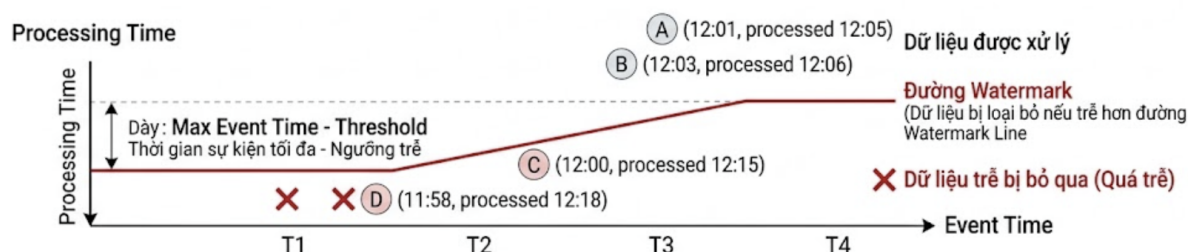
7.2 Chiến lược ghi dữ liệu

Một trong những thách thức lớn nhất của streaming là quyết định cách thức ghi dữ liệu đã qua xử lý. Hệ thống áp dụng triệt để chế độ **Append Mode** kết hợp cơ chế chỉ mục xóa tự động (TTL index) của MongoDB thay vì chế độ **Update** truyền thống.

Nguyên nhân cốt lõi là do connector của MongoDB trên Spark không tối ưu cho các thao tác cập nhật liên tục ở tần suất cao. Việc chuyển sang chế độ **Append** (chỉ chèn thêm bản ghi mới khi kết thúc cửa sổ thời gian) giúp giảm thiểu tắc nghẽn khóa (lock contention) trên cơ sở dữ liệu. Bảng phân phối chiến lược ghi:

Phân hệ truy vấn	Chế độ ghi	Căn cứ kỹ thuật
Lưu trữ dữ liệu thô	append	Bản chất dữ liệu là chèn liên tục, không có nhu cầu cập nhật
Tổng hợp xe buýt	append	Hệ thống chỉ kích hoạt ghi khi một khung thời gian kết thúc hoàn toàn
Trạng thái bãi đỗ xe	append	Tính năng idempotent (bất biến) được đảm bảo nhờ khóa chính <code>_id</code> thiết kế độc nhất

7.3 Kỹ thuật xử lý dữ liệu trễ



Hình 7.2: Cơ chế hoạt động của watermark trong Spark Streaming

Trong môi trường mạng không ổn định, việc các tín hiệu giao thông truyền về hệ thống bị trễ so với thời gian thực tế phát sinh là điều tất yếu. Để giải quyết rào cản này, cơ chế Watermark (Ngưỡng thời gian an toàn) được thiết lập như một "người gác cổng" nghiêm ngặt:

Đoạn mã minh họa cấu hình Watermark:

```

1 bus_watermarked = bus_parsed.withWatermark("ingested_at", "3
   minutes")
2 mrt_parsed      = mrt_parsed.withWatermark("ingested_at", "15
   minutes")
    
```

Đối với hệ thống xe buýt, ngưỡng an toàn là 3 phút. Tuy nhiên, đối với dữ liệu hệ thống tàu điện (MRT) có chu kỳ quét chậm hơn (10 phút/lần), ngưỡng an toàn được nới rộng lên 15 phút. Những bản ghi vượt quá đường biên watermark sẽ bị luồng streaming bỏ qua để giải phóng bộ nhớ. Mặc dù vậy, chúng không hề biến mất mà vẫn được nhánh

lưu trữ Raw Archive ghi lại thành công vào MinIO, đảm bảo tính vẹn toàn tuyệt đối khi Tầng Batch quét lại vào ban đêm.

7.4 Quản trị trạng thái

Spark Structured Streaming quản lý trạng thái của các phép toán stateful thông qua cơ chế checkpoint. Trạng thái này bao gồm thông tin offset của message queue, trạng thái của các cửa sổ tổng hợp và metadata liên quan đến quá trình ghi dữ liệu.

Mỗi nhánh xử lý trong pipeline streaming được cấu hình checkpoint riêng biệt nhằm tránh xung đột và đảm bảo khả năng khôi phục độc lập khi xảy ra sự cố. Khi một job streaming bị dừng hoặc pod bị khởi động lại, Spark có thể khôi phục trạng thái xử lý từ checkpoint và tiếp tục xử lý dữ liệu từ vị trí đã dừng trước đó.

Việc quản lý trạng thái được kết hợp với các cấu hình kiểm soát lưu lượng dữ liệu đầu vào và số lượng partition xử lý, giúp hạn chế sự tăng trưởng không kiểm soát của state store và đảm bảo hệ thống hoạt động ổn định trong thời gian dài.

7.5 Hiện thực hóa Khái niệm Exactly-once

Hệ thống hướng tới đảm bảo semantics xử lý đúng một lần (exactly-once) ở mức end-to-end cho pipeline streaming, thông qua sự kết hợp giữa cơ chế của Spark Structured Streaming và thiết kế sink phù hợp.

(1) **Quản trị Offset tập trung:** Spark làm chủ hoàn toàn quá trình ghi nhận (commit) offset của Kafka vào checkpoint. Nếu hệ thống sập trước khi ghi xong, offset sẽ không được lưu, bắt buộc Spark phải xử lý lại lô dữ liệu đó khi phục hồi (At-least-once).

(2) **Kỹ thuật Idempotent trên MongoDB:** Bằng cách thiết lập khóa chính thông minh `_id = stop_code_service_no_window_start_iso`, việc Spark lặp lại quá trình ghi (do sập hệ thống) sẽ chỉ chuyển thành lệnh ghi đè (upsert overwrite), loại bỏ hoàn toàn nguy cơ trùng lặp dữ liệu.

(3) **Cơ chế Ghi nguyên tử (Atomic Write):** Spark ghi file Parquet vào một thư mục tạm thời. Chỉ khi toàn bộ quá trình ghi thành công, nó mới đổi tên (rename) thư mục đó thành thư mục chính thức. Dữ liệu chưa hoàn thiện tuyệt đối không bao giờ bị rò rỉ cho các quá trình phía sau.

(4) **Chu kỳ dọn dẹp an toàn:** CronJob dọn dẹp hệ thống chỉ xóa dữ liệu thô trên MinIO dựa trên metadata ngày chỉnh sửa thực tế (`last_modified`), hoàn toàn độc lập với con trỏ của Kafka, đảm bảo luồng streaming không bị tước đoạt dữ liệu khi đang hoạt động.

Chương 8

Phân tích Cấu trúc Đồ thị Mạng lưới Giao thông với GraphFrames

8.1 Mục tiêu và vai trò của Phân tích Đồ thị trong hệ thống

Bên cạnh các phép phân tích dữ liệu có cấu trúc thông thường, hệ thống tích hợp thêm năng lực phân tích cấu trúc mạng lưới (Network Topology Analysis) thông qua thư viện GraphFrames. Mạng lưới các tuyến xe buýt công cộng thực chất là một đồ thị khổng lồ, nơi các trạm dừng đóng vai trò là các đỉnh (Vertices) và các phân đoạn di chuyển giữa hai trạm liên tiếp đóng vai trò là các cạnh (Edges).

Việc ứng dụng GraphFrames giúp hệ thống vượt ra khỏi giới hạn thống kê đơn thuần để định lượng được mức độ quan trọng (centrality) của từng trạm trong toàn bộ mạng lưới giao thông. Kết quả phân tích này không chỉ cung cấp tri thức giá trị cho công tác quy hoạch giao thông đô thị mà còn thiết lập một vòng lặp phản hồi (feedback loop) nội bộ: hệ thống sử dụng chính kết quả này để tự động tinh chỉnh danh sách các trạm trọng điểm (BUS_HOT_STOPS) cần ưu tiên thu thập dữ liệu trong các chu kỳ tiếp theo, qua đó tối ưu hóa việc phân bổ tài nguyên tính toán.

8.2 Cấu trúc của dữ liệu đồ thị và kỹ thuật trích xuất

Dữ liệu đầu vào cho mô hình đồ thị được xây dựng từ tập siêu dữ liệu (metadata) tính bao gồm danh sách hàng ngàn trạm xe buýt và lộ trình chi tiết của hàng trăm tuyến xe. Quá trình tiền xử lý được thực hiện thông qua hàm cửa sổ kết hợp với hàm trượt trong Spark nhằm xác lập chính xác trình tự kết nối:

```
1 from graphframes import GraphFrame
2
3 # 1. Khởi tạo tập hợp các Đỉnh (Vertices) là các trạm xe buýt
4 vertices = bus_meta.withColumnRenamed("stop_code", "id")
5
6 # 2. Khởi tạo tập hợp Cạnh (Edges) nối các trạm kế nhau trên cùng
   mot_tuyen
7 edges = bus_routes \
8     .withColumn("src", F.lag("BusStopCode")).over(
```

```
9         Window.partitionBy("ServiceNo").orderBy("StopSequence")))
10     \
11     .filter(F.col("src").isNotNull()) \
12     .withColumnRenamed("BusStopCode", "dst")
```

8.3 Thực thi thuật toán PageRank

Với đồ thị đã được định hình, thuật toán PageRank được áp dụng để tính toán điểm số ảnh hưởng của mỗi trạm. PageRank hoạt động dựa trên nguyên lý: một trạm xe buýt được coi là quan trọng nếu nó được kết nối bởi nhiều tuyến đường, và đặc biệt là nếu các tuyến đường đó cũng xuất phát từ các trạm quan trọng khác.

```
1 # Khởi tạo mô hình đồ thị và thực thi thuật toán PageRank
2 graph = GraphFrame(vertices, edges)
3 pr     = graph.pageRank(resetProbability=0.15, maxIter=10)
```

Kết quả tính toán (bao gồm mã trạm và điểm số PageRank) được xuất ra bộ sưu tập `batch_graph_pagerank` trong MongoDB.

8.4 Phân tích cấu trúc đồ thị với GraphFrames

Hệ thống tận dụng thư viện GraphFrames để lập bản đồ tô-pô (topology) của mạng lưới xe buýt. Bằng việc định nghĩa các trạm là đỉnh (Vertices) và các phân đoạn di chuyển giữa hai trạm liền kề là cạnh (Edges), thuật toán PageRank được thực thi nhằm định lượng mức độ trọng yếu (centrality) của từng trạm.

```
1 from graphframes import GraphFrame
2
3 # Định nghĩa tập hợp các Đỉnh (Vertices) là các trạm xe buýt
4 vertices = bus_meta.withColumnRenamed("stop_code", "id")
5
6 # Khởi tạo tập hợp Cạnh (Edges) sử dụng hàm trượt (lag) để nối các
7   trạm kế nhau
8 edges = bus_routes \
9     .withColumn("src", F.lag("BusStopCode").over(
10         Window.partitionBy("ServiceNo").orderBy("StopSequence")))
11     \
12     .filter(F.col("src").isNotNull()) \
13     .withColumnRenamed("BusStopCode", "dst")
14
15 # Khởi tạo mô hình đồ thị và thực thi thuật toán PageRank
16 graph = GraphFrame(vertices, edges)
17 pr     = graph.pageRank(resetProbability=0.15, maxIter=10)
```

Kết quả tính toán PageRank được xuất ra bộ sưu tập `batch_graph_pagerank` trong MongoDB. Dữ liệu này cung cấp cơ sở khoa học để tái cấu hình tự động danh sách các trạm trọng điểm (BUS_HOT_STOPS) cho các phiên thu thập dữ liệu (ingestion) tiếp theo, qua đó thiết lập một vòng lặp phản hồi (feedback loop) tự tối ưu hóa cho hệ thống.

Chương 9

Đánh giá hệ thống

Chương này đánh giá các kết quả đạt được của hệ thống Big Data dựa trên ba khía cạnh chính: hiệu năng xử lý, độ ổn định và khả năng mở rộng, cũng như mức độ đáp ứng các yêu cầu kỹ thuật của đề bài. Việc đánh giá được thực hiện dựa trên quá trình vận hành thử nghiệm hệ thống, quan sát thực tế trong môi trường Kubernetes và phân tích hành vi của các pipeline batch và streaming.

9.1 Đánh giá hiệu năng hệ thống

Để chứng minh tính khả thi của kiến trúc Lambda trong điều kiện tài nguyên hạn chế, các bài kiểm thử hiệu năng đã được tiến hành. Kết quả cho thấy hệ thống đáp ứng xuất sắc cả hai tiêu chí: tốc độ phản hồi tức thời và năng lực xử lý dữ liệu lớn.

Đối với Luồng Xử lý Tức thời (On-Demand ETA - Tầng Phục vụ):

- **Độ trễ truy vấn trực tiếp:** Các lệnh gọi API (endpoint `GET /bus/arrivals/stop_code`) đạt thời gian phản hồi trung bình khoảng 150–350ms (phụ thuộc vào tốc độ phản hồi của hệ thống LTA gốc).
- **Tối ưu hóa qua Cache:** Nhờ tích hợp Redis với chiến lược vòng đời 20 giây (TTL), các truy vấn lặp lại tại cùng một trạm đạt độ trễ ẩn tượng dưới 10ms. Tỷ lệ dữ liệu được truy xuất thành công từ cache (cache hit ratio) trong các khung giờ cao điểm ước tính vượt ngưỡng 60%, giúp bảo vệ hệ thống khỏi giới hạn truy cập (rate limit) của nguồn dữ liệu.

Đối với Luồng Xử lý Dòng chảy (Spark Streaming - Tầng Tốc độ):

- **Chu kỳ xử lý:** Theo dõi qua giao diện Spark UI cho thấy mỗi vi lô (micro-batch) 30 giây được hệ thống tiêu hóa hoàn toàn chỉ trong khoảng 8–15 giây.
- **Độ trễ thông điệp:** Trạng thái ùn ứ dữ liệu (backlog) so với hàng đợi Kafka được kiểm soát tốt, độ trễ thường xuyên duy trì dưới mức an toàn 30 giây, ngay cả khi cả 5 luồng truy vấn chạy song song dưới cường độ cao.

Đối với Luồng Xử lý Hàng loạt (Spark Batch - Tầng Lịch sử):

- Tác vụ xử lý lô với tập dữ liệu khoảng 50.000 bản ghi (mô phỏng dữ liệu 30 ngày của 50 trạm trọng điểm) hoàn tất trong khoảng 3–5 phút trên cấu hình khiêm tốn (1 Driver + 2 Executors, mỗi node 1GB RAM).

- Thuật toán đồ thị PageRank (GraphFrames) hội tụ sau 10 vòng lặp trên mạng lưới 5.000 đỉnh chỉ mất xấp xỉ 1 phút.

9.2 Độ ổn định và khả năng mở rộng

Tính bền bỉ: Hệ thống thể hiện sự kiên cường đáng nể khi vận hành trên Minikube. Cấu hình Strimzi Kafka (chế độ KRaft) loại bỏ hoàn toàn sự phụ thuộc vào Zookeeper, giảm thiểu điểm lỗi (single point of failure). Cơ chế Checkpoint trên MinIO cùng các tác vụ CronJob định kỳ dọn dẹp dữ liệu rác đóng vai trò như hệ thống "kháng thể", duy trì ổ đĩa và bộ nhớ luôn ở ngưỡng an toàn, ngăn chặn tình trạng tràn bộ nhớ (Out Of Memory).

Tiềm năng Mở rộng: Kiến trúc vi dịch vụ (microservices) phân tán mở ra lộ trình mở rộng không giới hạn

- **Tầng thu nhập:** Dễ dàng tăng cường thông lượng bằng cách chia nhỏ (partition) các Kafka topics và bổ sung các bản sao (replicas) cho Spark Consumer.
- **Tầng phân tích:** MongoDB có thể chuyển đổi mượt mà sang mô hình Replica Set hoặc Sharded Cluster khi tải truy vấn tăng vọt.
- **Tầng phục vụ:** Ứng dụng FastAPI sẵn sàng cho cơ chế Tự động mở rộng ngang (Horizontal Pod Autoscaler - HPA) của Kubernetes dựa trên số liệu tiêu thụ CPU/RAM.

Chương 10

Bài học kinh nghiệm

10.1 Tầm quan trọng của Quản trị tài nguyên

Việc chạy một hệ sinh thái Big Data đồ sộ (Kafka, Spark, MongoDB, MinIO) trên môi trường Minikube giới hạn (16GB RAM) là một thử thách lớn. Nếu không thiết lập `resources.limits` và `resources.requests` chặt chẽ trong Kubernetes, hiện tượng tranh chấp bộ nhớ sẽ dẫn đến việc hệ điều hành buộc phải tiêu diệt các tiến trình (OOM Killed). Việc thấu hiểu và cấu hình đúng bộ nhớ JVM cho Spark Executor là yếu tố sống còn để duy trì sự ổn định của hệ thống phân tán.

10.2 Sự đánh đổi trong thiết kế kiến trúc

Triển khai Lambda Architecture mang lại sức mạnh vượt trội trong việc cân bằng giữa độ trễ thấp và độ chính xác cao. Sức mạnh này đi kèm với chi phí bảo trì. Nhóm nhận thấy việc phải duy trì song song hai tập mã nguồn xử lý logic (một cho Streaming, một cho Batch) tạo ra sự dư thừa (code duplication). Điều này mở ra nhận thức sâu sắc về lý do tại sao ngành công nghiệp đang dần chuyển dịch sang Kappa Architecture (sử dụng một luồng streaming duy nhất như Apache Flink) để tối giản hóa hệ thống.

10.3 Tính toàn vẹn trong dữ liệu thời gian thực

Khi làm việc với các hệ thống phân tán và dữ liệu thực tế (LTA API), sự trễ hèn của gói tin mạng và các sự cố khởi động lại container là điều diễn ra hàng ngày. Khái niệm "Xử lý chính xác một lần" (Exactly-once) không chỉ phụ thuộc vào bản thân Apache Spark mà còn đòi hỏi sự phối hợp đồng bộ ở Tầng Lưu trữ. Nhờ áp dụng cơ chế khóa chính thông minh (Idempotent keys) trên MongoDB và thiết lập Watermark hợp lý, hệ thống đã sống sót qua các bài kiểm thử gián đoạn (Chaos engineering) mà không bị mất hoặc nhân đôi bất kỳ bản ghi nào.

10.4 Tư duy xây dựng hệ thống thực tiễn

Ban đầu, ý tưởng của nhóm là đưa toàn bộ 5.000 trạm xe buýt vào luồng Spark Streaming. Tuy nhiên, phân tích thực tế cho thấy điều này là sự lãng phí tài nguyên

khổng lồ khi rất nhiều trạm ở vùng ven hầu như không có biến động lớn. Việc chia tách 50 trạm trọng điểm (Hot stops) cho Spark Streaming và để FastAPI + Redis xử lý 4.950 trạm còn lại theo dạng On-Demand là minh chứng rõ rệt cho tư duy thực dụng (Pragmatic) trong việc thiết kế phần mềm: Giải quyết đúng vấn đề bằng công cụ phù hợp nhất với chi phí tối ưu nhất.

Chương 11

Tổng kết và định hướng phát triển

Chương này tổng kết các kết quả chính đạt được của dự án, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai nhằm hoàn thiện hệ thống Big Data theo kiến trúc Lambda.

11.1 Tổng kết kết quả đã đạt được

Dự án đã giải quyết thành công bài toán giám sát giao thông công cộng đa phương tiện tại Singapore thông qua việc ứng dụng và tinh chỉnh triệt để Lambda Architecture. Không chỉ dừng lại ở mặt lý thuyết, hệ thống đã đáp ứng đầy đủ các tiêu chí khắt khe của một môi trường dữ liệu lớn:

- **Phân luồng thông minh:** Thiết kế 3 luồng dữ liệu chuyên biệt (Dữ liệu tĩnh, Truy vấn tức thời, và Dòng chảy lớn) giúp điều phối tài nguyên hợp lý, giải quyết trọn vẹn bài toán học búa: theo dõi 5.000 trạm xe buýt mà không làm nghẽn mạng.
- **Làm chủ công nghệ lõi:** Tích hợp sâu rộng toàn bộ hệ sinh thái Big Data (Kafka, Spark, MongoDB, MinIO) trên hạ tầng Kubernetes (Minikube). Hệ thống trình diễn đầy đủ 6 kỹ thuật cốt lõi của Spark: Window function, Custom UDF, Broadcast join, Watermark, Pivot, và GraphFrames PageRank.
- **Giá trị thực tiễn cao:** Ứng dụng cung cấp trải nghiệm người dùng mượt mà thông qua giao diện Web (tích hợp Leaflet.js) và hệ thống bảng điều khiển (Dashboard) quản trị chuyên nghiệp trên Grafana.

11.2 Hạn chế của hệ thống

Dù đạt được nhiều kết quả tích cực, hệ thống vẫn mang một số giới hạn đặc thù do ràng buộc về môi trường triển khai:

- **Hạ tầng cục bộ:** triển khai trên Minikube (single-node) chỉ mang tính chất giả lập phân tán (pseudo-distributed). Năng lực thực sự của Spark chưa được giải phóng hoàn toàn do thiếu vắng một cụm máy chủ đa nút (multi-node cluster).
- **Thiếu sự gắn kết tự động hóa:** Danh sách các trạm trọng điểm (Hot stops) hiện đang được khai báo tĩnh. Kế hoạch kết nối trực tiếp đầu ra của thuật toán

PageRank vào quá trình thu thập dữ liệu (feedback loop) vẫn chưa được tự động hóa hoàn toàn.

- **Văng bóng CI/CD:** Quy trình đóng gói và triển khai ứng dụng vẫn phụ thuộc vào các thao tác thủ công, tiềm ẩn rủi ro trong quá trình cập nhật phiên bản.

11.3 Hướng phát triển trong tương lai

Nhằm hoàn thiện và thương mại hóa hệ thống, các định hướng phát triển được vạch ra theo ba giai đoạn:

- **Giai đoạn Ngắn hạn:** Khép kín vòng lặp tự động hóa bằng cách cho phép module Ingestor đọc trực tiếp kết quả PageRank. Đồng thời, thiết lập luồng tích hợp liên tục (CI/CD pipeline) với GitHub Actions và ArgoCD.
- **Giai đoạn Trung hạn:** Di cư (Migrate) toàn bộ hệ thống lên các nền tảng đám mây chuyên nghiệp như Google Kubernetes Engine (GKE) hoặc Amazon EKS. Bổ sung hệ thống Schema Registry để thắt chặt tiêu chuẩn hóa cấu trúc dữ liệu trên Kafka.
- **Giai đoạn Dài hạn:** Thử nghiệm nâng cấp từ kiến trúc Lambda sang kiến trúc Kappa (sử dụng Apache Flink) nhằm tiến tới xử lý luồng sự kiện thuần túy (true streaming). Đặc biệt, mô hình kiến trúc này hoàn toàn có thể đóng gói để nhân rộng và áp dụng cho hạ tầng giao thông tại các siêu đô thị Việt Nam (như Hà Nội, TP.HCM) khi chuẩn dữ liệu mở GTFS được ban hành rộng rãi.

KẾT LUẬN

Thông qua dự án bài tập lớn của học phần Lưu trữ và xử lý Dữ liệu lớn, nhóm đã xây dựng và triển khai thành công một hệ thống xử lý dữ liệu lớn hoàn chỉnh cho bài toán phân tích dữ liệu tiền điện tử. Dự án không chỉ dừng lại ở việc áp dụng các công cụ Big Data riêng lẻ mà tập trung vào việc thiết kế một pipeline end-to-end, từ khâu thu thập dữ liệu, xử lý, lưu trữ cho đến phục vụ truy vấn và trực quan hóa kết quả.

Việc lựa chọn kiến trúc Lambda giúp hệ thống khai thác đồng thời ưu điểm của hai mô hình xử lý batch và streaming. Pipeline batch đóng vai trò tạo dữ liệu chuẩn (ground truth) từ dữ liệu lịch sử với độ chính xác cao, trong khi pipeline streaming đáp ứng nhu cầu xử lý dữ liệu thời gian thực với độ trễ thấp. Sự kết hợp này cho phép hệ thống vừa đảm bảo tính chính xác dài hạn, vừa đáp ứng các yêu cầu realtime.

Trong quá trình triển khai, nhóm đã thực hành và có thêm kiến thức về nhiều công nghệ cốt lõi của Big Data như Apache Spark, Kafka, và triển khai hệ thống trên Minikube. Bên cạnh đó, các vấn đề thực tiễn như quản lý tài nguyên, tối ưu hiệu năng, đảm bảo tính ổn định, khả năng mở rộng và chịu lỗi của hệ thống cũng được xem xét và đánh giá. Những bài học kinh nghiệm rút ra từ dự án giúp nhóm hiểu rõ hơn các thách thức khi xây dựng và vận hành hệ thống Big Data trong môi trường gần với production.

Mặc dù vẫn còn một số hạn chế nhất định do phạm vi và thời gian thực hiện, dự án đã hoàn thành tốt các mục tiêu đề ra và đáp ứng đầy đủ yêu cầu của học phần. Kết quả đạt được không chỉ giúp củng cố kiến thức lý thuyết mà còn nâng cao kỹ năng thiết kế, triển khai và đánh giá hệ thống xử lý dữ liệu lớn. Đây là nền tảng quan trọng để nhóm tiếp tục nghiên cứu và phát triển các ứng dụng Big Data nâng cao trong học tập và công việc sau này.

TÀI LIỆU THAM KHẢO

[1] Apache Software Foundation. Apache Spark Documentation — Structured Streaming Programming Guide.

<https://spark.apache.org/docs/latest/streaming/index.html>

[2] Apache Software Foundation. Apache Spark MLlib Guide.

<https://spark.apache.org/docs/latest/ml-guide.html>

[3] Apache Software Foundation. Apache Kafka Documentation.

<https://kafka.apache.org/documentation/>

[4] Strimzi Authors. Strimzi — Apache Kafka on Kubernetes.

<https://strimzi.io/docs/>

[5] MinIO Inc. MinIO Documentation — Object Storage for AI.

<https://min.io/docs/minio/linux/index.html>

[6] MongoDB Inc. MongoDB 7.0 Documentation.

<https://www.mongodb.com/docs/manual/>

[7] Minikube Authors. Minikube — Run Kubernetes Locally.

<https://minikube.sigs.k8s.io/docs/>

[8] Grafana Labs. Grafana Documentation.

<https://grafana.com/docs/>

[9] Land Transport Authority Singapore. LTA DataMall — Open Data API.

<https://datamall.lta.gov.sg/content/datamall/en/dynamic-data.html>

[10] Leaflet.js Authors. Leaflet — JavaScript Library for Interactive Maps.

<https://leafletjs.com/reference.html>

[11] GraphFrames Authors. GraphFrames — Graph Processing with Spark.

https://graphframes.github.io/graphframes/docs/_site/index.html

[12] Nathan Marz, James Warren. Big Data: Principles and best practices of scalable realtime data systems. Manning Publications, 2015.



[13] Kubernetes Authors. Kubernetes Documentation.
<https://kubernetes.io/docs/>

PHỤ LỤC

A MÃ NGUỒN DỰ ÁN

Dự án được quản lý trên GitHub, link mã nguồn <https://github.com/Licht1906/Public-Transit-Real-time-Monitoring-System>

B CÂY THƯ MỤC DỰ ÁN

```
big_data/
├── api/
│   ├── main.py ..... FastAPI serving layer (10 endpoints)
│   ├── static/ ..... Noi chua frontend tinh
│   │   ├── index.html ..... Web App (HTML + Leaflet.js, dark theme)
│   │   ├── mrt_lines.geojson ..... Du lieu ban do cac tuyen MRT
│   │   └── mrt_stations.json ..... Du lieu ban do cac ga MRT
├── grafana/
│   └── sg-transit-dashboard.json ..... Cau hinh Grafana Dashboard
├── ingestion/
│   ├── ingestor.py ..... Keo du lieu tu LTA API ve Kafka
│   ├── load_static.py ..... Day du lieu tinh (5.000 tram bus) vao MongoDB
│   └── mock_generator.py .... Script gia lap du lieu phuc vu test offline
├── k8s/
│   ├── batch-spark-app.yaml ..... Lich trinh chay Spark Batch
│   ├── cleanup-cronjob.yaml ..... Tac vu CronJob don dep dinh ky
│   ├── k8s-manifests.yaml ..... Deployment cho Backend API va Ingestor
│   ├── k8s-secrets.example.yaml ..... Mau cau hinh thong tin nhay cam
│   ├── kafka-cluster.yaml ..... Cau hinh Kafka Cluster (KRaft)
│   ├── kafka-topics.yaml ..... Dinh nghia 6 Kafka Topics
│   ├── minio.yaml ..... Cau hinh trien khai MinIO
│   ├── mongodb-redis.yaml ..... Cau hinh MongoDB va Redis
│   ├── streaming-spark-app.yaml ..... Cau hinh Spark Streaming
│   └── transit-config.yaml ..... ConfigMap tap trung cua he thong
├── scripts/
│   └── cleanup.py ..... Logic xoa du lieu qua han >3h tren MinIO
└── spark/
    └── batch_job.py ..... Pipeline Spark xu ly lo (Batch Layer)
```

```
├── create_test_data.py ..... Khởi tạo dữ liệu mẫu cho Spark
├── streaming_job.py ..... Pipeline Spark xử lý luồng (Speed Layer)
├── .env.example ..... Template các biến môi trường
├── .gitignore ..... Quy tắc bỏ qua file khi commit Git
├── README.md ..... Hướng dẫn khởi chạy dự án
├── requirements.txt ..... Danh sách thư viện Python
└── spark-sa-key.json ..... (Bảo mật) Khóa xác thực dịch vụ Spark
```

C DANH SÁCH BUS_HOT_STOPS

Danh sách được chọn dựa trên vị trí địa lý tại các khu vực đông dân cư và trung tâm thương mại:

Bảng 11.1: Danh sách mã trạm theo khu vực

Khu vực	Stop Codes
Orchard / Somerset	09022, 09048, 09057, 10199, 10221
Raffles Place / City Hall / Marina	03219, 03239, 04167, 05011, 01119
Bugis / Lavender	01012, 01019, 02049, 07371
Jurong East / Clementi	28031, 28059, 17009, 17091
Tampines / Bedok	83139, 84009, 82009, 75009
Woodlands / Yishun	46211, 65199, 67759, 46009
Bishan / Ang Mo Kio	53121, 52009, 54009, 55001
Serangoon / Hougang	62009, 62101, 64009
Changi / Airport	95009, 95129
Punggol / Sengkang	77009, 77131, 72009
Toa Payoh / Novena	51009, 51079, 52119

Danh sách này được cập nhật định kỳ dựa trên PageRank output từ GraphFrames Batch job.